

---

# Software Requirement Specifications

for

LawPal : A Retrieval Augmented Generation  
Based System for Enhanced Legal Accessibility  
in India.

**LAWPAL**

Version 1.0 approved

Prepared by:

ADITHYA KANDALA

AMGOTH VIJAY KUMAR

VUTHULURI AMARESHWAR

ARE SAINATH

AQEEL AHMED

8/26/2025

# Table of Contents

|                                        |    |
|----------------------------------------|----|
| Table of Contents.....                 | 2  |
| 1. Introduction.....                   | 3  |
| 1.1 Purpose.....                       | 3  |
| 1.2 Intended Audience.....             | 3  |
| 1.3 Product Scope.....                 | 3  |
| 2. Overall Description.....            | 4  |
| 2.1 Product Perspective.....           | 4  |
| 2.2 Product functions.....             | 4  |
| 2.3 User Classes.....                  | 5  |
| 2.4 Operating Environment.....         | 5  |
| 3. External Interface Requirement..... | 7  |
| 3.1 User Interfaces.....               | 7  |
| 3.2 Hardware Interfaces.....           | 7  |
| 3.3 Software Interfaces.....           | 8  |
| 4. System Features.....                | 10 |
| 4.1 Vakeel-AI.....                     | 10 |
| 4.2 User Authentication.....           | 11 |
| 4.3 User Account Management.....       | 11 |
| 4.4 Query History and Revision.....    | 12 |
| 4.5 Webscraper.....                    | 13 |

# 1. Introduction

## 1.1 Purpose

The purpose of this API is to make legal knowledge accessible and easy-to-understand for the users in India by integrating legal resources and AI-powered query system to clarify their doubts.

## 1.2 Intended Audience

The intended audience of this API can be anybody from regular citizens who may want to simply clarify their queries in a simpler manner to those directly working in the law departments such as lawyers or government officials.

## 1.3 Product Scope

The API is built by using software such as FAISS, deepseek for encoding, Langchain for embedding, React for frontend and Node,Express for backend. The product will be able to take in the user query and retrieve data from a dataset to obtain context with which it will generate the output.

## 2. Overall Description

### 2.1 Product Perspective

Our API is a standalone project that is not a part of any other API or project but it is inspired off of an already published project model called LegalEase. Our model is heavily based on Legalease`s Technology stack and concepts.

### 2.2 Product Functions

The software product provides the following major functions:

1. AI-Powered Legal Assistance (Vakeel):
  - Acts as the central feature of the system.
  - Allows users to enter queries in natural language.
  - Uses a custom-trained AI model to understand queries, retrieve relevant legal information, and present clear and simplified responses.
  - Supports detailed explanations and concise summaries for easier understanding.
2. User Authentication (Login/Logout):
  - Provides secure login/logout mechanisms for users.
  - Ensures only authorized users can access the system.
3. User Account Management:
  - Allows administrators to create, update, and delete user accounts.
  - Assigns roles (citizen, student, official) with different levels of access.
4. Query Handling and History:
  - Enables users to submit queries and receive accurate responses.
  - Stores query history for later review.
  - Provides options to clear or export query history.

## 5. Error Handling and Notifications:

- Displays user-friendly error messages when invalid inputs are detected.
- Provides notifications for system updates, outages, or account-related issues.

## 2.3 User Classes

For this API, the intended user classes include a wide range of individuals, from regular citizens to officials working in law-related departments.

- Regular Citizens: May use the application to clarify doubts regarding laws or legal procedures in simple language.
- Students (Law Department): Can utilize the application as a learning tool for study, research, or quick reference.
- Law Officials and Professionals: May use the application for further clarification, quick access to information, or fast revision in their professional work.

## 2.4 Operating Environment

The API will operate in a standard web-based environment and must be compatible with commonly used platforms and tools. The operating environment includes:

- Hardware Platform:
  - Standard server-grade hardware or cloud-hosted infrastructure with at least 4-core processors, 8 GB RAM, and sufficient storage based on database size and traffic.

- Client-side devices may include desktops, laptops, tablets, and smartphones capable of running modern web browsers.
- Operating System:
  - Server: Linux (Ubuntu 20.04 LTS or later) or Windows Server 2019 and above.
  - Client: Any OS capable of running a supported browser (Windows 10/11, macOS, Linux, Android, iOS).
- Software Components:

Backend: Node.js (v18.x or later), Express.js framework.

  - Database: MongoDB (v6.x or later).
  - Frontend: React.js.
  - AI Integration: A custom trained AI model called VAKEEL.
  - Other Dependencies: .
  - Backend: Node.js (v18.x or later), Express.js framework.

## 3. External Interface Requirements

### 3.1 User Interfaces

The system will provide a web-based graphical user interface (GUI) that allows users to interact with the API in a simple and intuitive manner. The logical characteristics of the user interface include:

- General Layout and Style:
  - A consistent design will be followed across all screens, based on modern UI standards (responsive design, minimalistic layout, and accessibility compliance).
  - The interface will adopt a standard product family style guide (consistent colors, fonts, and button styles).
  - A navigation bar will be present on all screens for easy access to major functions.
- User Interface Components Needed:
  - Login/Authentication Screen – For user login and role-based access.
  - Dashboard Screen – Displays quick access to common queries and user options.
  - Query Input Screen – Allows users to type questions/doubts regarding laws and displays AI-generated responses and references.

### 3.2 Hardware Interfaces

The system will interact with both server-side and client-side hardware components. The logical and physical characteristics of these interfaces are as follows:

**Server-Side Hardware Interfaces:** The software will operate on standard server hardware or cloud-hosted infrastructure.

- Minimum requirements: Quad-core CPU, 8 GB RAM, and storage capacity sufficient for database growth and user traffic.
- Network interface cards (NICs) will be used to enable communication between the server and client devices via the internet.

#### Client-Side Hardware Interfaces:

- End users may access the system through desktops, laptops, tablets, or smartphones.
- Supported input devices include keyboard, touchscreens, and pointing devices.
- Supported output devices include monitors, mobile screens, and speakers (for accessibility features such as screen readers).

### 3.3 Software Interfaces

The system will interface with multiple software components to provide its intended functionality. These interfaces are defined as follows:

- **Operating System Interfaces:**

- The backend API will run on **Linux (Ubuntu 20.04 LTS or later)** or **Windows Server 2019 and above**.
- Standard OS-level services such as process management, networking, and file system access will be utilized.

- **Database Interfaces:**

- **MongoDB (v6.x or later)** will serve as the primary database for storing user details, queries, responses, and logs.
- The backend will interact with MongoDB using the **Mongoose ODM (Object Data Modeling) library**.
- Data exchanged:



- Incoming: User profiles, queries, and authentication credentials.
- Outgoing: AI-generated responses, stored history, and administrative reports.
- **Backend Framework Interfaces:**
  - **Node.js (v18.x or later)** will serve as the runtime environment.
  - **Express.js** will provide routing, middleware management, and RESTful API endpoint handling.
- **Frontend Interfaces:**
  - The frontend will be implemented in **React.js** and will interact with the backend API using **HTTP/HTTPS requests**.
  - Data exchanged:
    - Incoming: User input (queries, login data).
    - Outgoing: Results, clarifications, and status/error messages.
- **AI Model Integration:**
  - The AI we will be integrating will be a custom trained model specifically built for law related queries that follows RAG using FAISS and Deepseek Encoding.

## 4. System Features

### 4.1 Vakeel – AI-Powered Legal Assistant

#### 4.1.1 Description and Priority

Vakeel is the core component of the system, a custom-trained AI model designed to provide legal information, clarification, and assistance to users. It leverages natural language processing and FAISS similarity search to understand queries, retrieve relevant legal data, and present it in simplified, user-friendly language.

- Priority: Very High (primary focus of the project).

#### 4.1.2 Stimulus/Response Sequences

- User Action: A user submits a query (e.g., *“What is the punishment for cheque bounce in India?”*).
- System Response:
  1. The query is sent to Vakeel.
  2. Vakeel interprets the query, searches the legal database using FAISS, and generates a response.
  3. The response is sent back to the user in simple, understandable language.

#### 4.1.3 Functional Requirements

- REQ-VAKEEL-1: The system shall allow users to submit queries in natural language.
- REQ-VAKEEL-2: Vakeel shall analyze the query and provide contextually accurate answers.
- REQ-VAKEEL-3: Vakeel shall generate both detailed explanations and simplified summaries based on user needs.

- REQ-VAKEEL-4: The system shall ensure query responses are delivered in less than 3 seconds under normal load.
- REQ-VAKEEL-5: If the model cannot find relevant information, it shall notify the user politely and suggest alternatives.

## 4.2 User Authentication (Login/Logout)

### 4.2.1 Description and Priority

This feature enables secure access to the system by ensuring that only authorized users (citizens, students, or officials) can log in.

- Priority: High.

### 4.2.2 Stimulus/Response Sequences

- User Action: User enters login credentials.
- System Response: The system verifies credentials, grants/denies access, and establishes a session.

### 4.2.3 Functional Requirements

- REQ-AUTH-1: The system shall allow users to log in using a username/email and password.
- REQ-AUTH-2: The system shall allow users to log out and terminate sessions.
- REQ-AUTH-3: The system shall enforce secure password policies.
- REQ-AUTH-4: Invalid login attempts shall be logged and limited to prevent brute force attacks.

## 4.3 User Account Management

### 4.3.1 Description and Priority

This feature allows admins to create and manage user accounts and roles (citizen, student, official).

- Priority: Medium.

#### 4.3.2 Stimulus/Response Sequences

- User Action: Admin creates a student account.
- System Response: The system generates login credentials and notifies the student.

#### 4.3.3 Functional Requirements

- REQ-UAM-1: The system shall allow admins to create, update, and delete user accounts.
- REQ-UAM-2: The system shall assign roles with appropriate access levels.
- REQ-UAM-3: The system shall notify users of new credentials securely.

### 4.4 Query History and Revision

#### 4.4.1 Description and Priority

This feature enables users to view, revisit, and manage their previous queries and results.

- Priority: Medium.

#### 4.4.2 Stimulus/Response Sequences

- User Action: A user checks their query history.
- System Response: The system retrieves and displays past queries and Vakeel's responses.

#### 4.4.3 Functional Requirements

- REQ-QHR-1: The system shall store user queries and responses in a secure database.
- REQ-QHR-2: Users shall be able to view their history upon login.
- REQ-QHR-3: The system shall allow users to clear or export their history.

## 4.5 Webscraper

### 4.5.1 Description and Priority

The Webscraper is a background subsystem responsible for automatically collecting legal documents, judgments, circulars, and statutory information from authorized public sources. It extracts text, cleans the content, removes duplicates, and prepares documents for embedding and indexing in the FAISS/Chroma store. This ensures continuous, up-to-date legal information for the Vakeel system.

Priority: High, as it directly impacts the freshness and completeness of the knowledge base used by the AI.

### 4.5.2 Stimulus/Response Sequences

Stimulus:

The system scheduler triggers a scrape operation, or an administrator manually submits a URL for ingestion.

System Response:

1. The Webscraper fetches the webpage content.
2. The system extracts text, metadata, and relevant legal information.

3. The system checks for duplicates using hashing and embedding similarity.
4. If new, the document is stored in MongoDB and sent for embedding.
5. The vector representation is indexed in FAISS/Chroma.
6. The system updates the document status and logs results.